Module 4: Node.js, Express & TypeORM › Section 13: Sessions, Authentication & Security › 13-02 — Authentication

Authentication — Passport.js, JWT, and Auth Guards

Document Information

Section	13 — Sessions, Authentication & Security
Subtopic	02 — Authentication
Estimated Duration	2 – 2.5 hours
Tags	Theory Practical TypeScript
Topics Covered	Password hashing with `bcrypt`, TypeORM User entity, Passport.js local strategy, session fixation & `req.session.regenerate()`, JWT with `jsonwebtoken`, auth guard middleware, session-based vs token-based auth

Learning Objectives — This Subtopic

- Explain the difference between session-based and token-based authentication, and identify scenarios where each is appropriate.
- Design a TypeORM `User` entity with correctly typed fields and integrate `bcrypt` for safe password storage.
- Configure Passport.js with the Local strategy to authenticate users against a TypeORM-backed database, and wire it into the session lifecycle.
- Prevent session fixation attacks by calling `req.session.regenerate()` at the correct point in the login flow.

- Issue, sign, and verify JSON Web Tokens using `jsonwebtoken`, and implement a reusable JWT auth guard middleware in TypeScript.
- Write route-level auth guard middleware that protects endpoints under both session-based and JWT-based auth strategies.

1. Authentication Concepts and Strategy Choice

Authentication is the process of verifying that a user is who they claim to be. It is distinct from *authorisation*, which determines what an authenticated user is allowed to do. This subtopic covers authentication exclusively.

There are two dominant strategies for maintaining authenticated state in an HTTP application:

Session-Based Authentication

- Server stores session data (including user identity) after login — covered in 13-01.
- Client holds only a session ID cookie.
- Server can invalidate a session instantly by deleting it from the store.
- Natural fit for traditional web apps with server-rendered pages.
- Requires a shared, persistent session store (Redis) when scaling horizontally.

Token-Based Authentication (JWT)

- Server issues a signed token after login; no server-side state is kept.
- Client stores the token (commonly in `Authorization` header or `localStorage`).
- Stateless — any server instance can verify the token without shared storage.
- Natural fit for REST APIs consumed by SPAs or mobile clients.
- Revocation is complex without a token blocklist (which re-introduces state).

This document implements both approaches against the same TypeORM User entity so you can choose the right tool for a given project.

2. Project Setup

Install the packages required for this subtopic. This builds on the `express-session` and `cookie-parser` setup from 13-01.

```
npm install passport passport-local bcrypt jsonwebtoken
```

```
npm install --save-dev @types/passport @types/passport-local @types/bcrypt @types/jsonwebtoken
```

The project structure for the examples in this document:

```
auth-demo/ ├── src/ │ ├── app.ts ← Express setup, middleware registration │ ├── data-source.ts ← TypeORM DataSource configuration │ ├── session.d.ts ← Session type augmentation (from 13-01) │ ├── entities/ │ │ └── User.ts ← TypeORM User entity │ ├── auth/ │ │ ├── passport.ts ← Passport strategy configuration │ │ ├── jwt.ts ← JWT sign/verify helpers │ │ └── guards.ts ← Auth guard middleware functions │ └── routes/ │ ├── auth.routes.ts ← /register, /login, /logout, /me │ └── protected.routes.ts ← Routes behind auth guards ├── tsconfig.json └── package.json
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true
  },
  "include": ["src"]
}
```

> **Key Insight:** `emitDecoratorMetadata` and `experimentalDecorators` are required for TypeORM decorators (`@Entity` , `@Column` , etc.) to function correctly. These were set up in Section 10 when TypeORM was introduced.

3. The User Entity and Password Hashing

3.1 Designing the User Entity

Before any authentication logic can run, you need a way to persist users. The `User` entity maps to a `users` table and holds the minimum fields required for local authentication: an email address and a hashed password.

src/entities/User.ts

```
import {
    Entity,
    PrimaryGeneratedColumn,
    Column,
    CreateDateColumn,
    UpdateDateColumn,
} from 'typeorm';

@Entity('users')
export class User {
    @PrimaryGeneratedColumn()
    id!: number;

    @Column({ unique: true, length: 254 })
    email!: string;

    // Stores the bcrypt hash, not the plaintext password.
    // The field is intentionally not selected by default in queries
    // that return user data to clients.
    @Column({ select: false })
    passwordHash!: string;

    @Column({ default: false })
    isActive!: boolean;

    @CreateDateColumn()
    createdAt!: Date;

    @UpdateDateColumn()
    updatedAt!: Date;
}
```

Key Insight: `@Column({ select: false })` causes TypeORM to omit `passwordHash` from the result set whenever you use the standard `find*` repository methods. This prevents accidental hash leakage — for example, an endpoint that returns user profile data will not inadvertently include the hash. When you *need* the hash (i.e., during login verification), you must explicitly request it using `addSelect` in a query builder call, as shown in Section 3.3.

3.2 Password Hashing with bcrypt

Storing a plaintext password is an absolute non-starter. Storing a fast hash (MD5, SHA-256) is almost as bad — these can be reversed using pre-computed rainbow tables and cracked rapidly with GPU brute force. **bcrypt** is designed specifically for password storage: it is computationally expensive by design (via a configurable work factor), includes a built-in random salt, and is slow to parallelise on GPUs.

Analogy — The Cost Factor as a Time Lock
Think of the bcrypt work factor (also called *cost* or *rounds*) as the number of times you have to turn a combination lock before it opens. For your application, turning it 10–12 times takes about 100–300 ms — imperceptible to a legitimate user logging in. For an attacker trying billions of passwords, that 100 ms per attempt makes a brute-force attack take centuries rather than hours.

Stage 1 Hash a password during registration and verify it during login.

`src/auth/password.ts`

```
import bcrypt from 'bcrypt';

const SALT_ROUNDS = 12;  // Work factor: increase as hardware improves

/**
 * Hashes a plaintext password. Call this once at registration time.
 * Never call it again for the same password — bcrypt generates a new
 * random salt each time, producing a different hash even for the same input.
 */
export async function hashPassword(plaintext: string): Promise<string> {
    return bcrypt.hash(plaintext, SALT_ROUNDS);
}

/**
 * Compares a plaintext candidate against a stored bcrypt hash.
```

```
 * Returns true only if the candidate matches.
 */
export async function verifyPassword(
    plaintext: string,
    hash: string
): Promise<boolean> {
    return bcrypt.compare(plaintext, hash);
}
```

Output:

```
// Quick test in ts-node REPL:
const hash = await hashPassword('correct-horse-battery-staple');
console.log(hash);
// $2b$12$eKh8Kn7VY3Rz4Y5B1x9Oj.Qr5Lg0YdO2f9wQ7J1sZ3oK8xFpN.lC

const ok = await verifyPassword('correct-horse-battery-staple', hash);
console.log(ok);   // true

const bad = await verifyPassword('wrong-password', hash);
console.log(bad);  // false
```

What is happening? `bcrypt.hash()` generates a random 128-bit salt, combines it with the plaintext, and runs the bcrypt algorithm for `2^SALT_ROUNDS` iterations. The salt is embedded in the output hash string — that is why `bcrypt.compare()` does not need the salt passed separately; it reads it directly from the stored hash.

Important: Never compare password hashes with `===`. Use `bcrypt.compare()` exclusively. Direct string comparison is vulnerable to timing attacks — the time taken to compare two strings varies with how many characters match, leaking information to an attacker. `bcrypt.compare()` uses a constant-time comparison internally.

3.3 DataSource and Repository Setup

src/data-source.ts

```
import { DataSource } from 'typeorm';
import { User } from './entities/User';

export const AppDataSource = new DataSource({
    type: 'postgres',
```

```
    host: process.env.DB_HOST ?? 'localhost',
    port: Number(process.env.DB_PORT ?? 5432),
    username: process.env.DB_USER ?? 'postgres',
    password: process.env.DB_PASS ?? 'postgres',
    database: process.env.DB_NAME ?? 'auth_demo',
    entities: [User],
    synchronize: process.env.NODE_ENV !== 'production',  // migrations in prod
    logging: false,
});

/** Returns the User repository with all standard methods available. */
export const getUserRepo = () => AppDataSource.getRepository(User);
```

> **Reminder from Section 10:** `synchronize: true` automatically aligns the database schema with your entity definitions on every application start. This is convenient in development but destructive in production, where schema changes must be managed via migrations (covered in Section 12). The conditional above enforces this distinction automatically based on `NODE_ENV`.

---

4. Session-Based Authentication with Passport.js

4.1 What Passport.js Does

Passport is authentication middleware for Express. It provides a consistent API (the `Strategy` pattern) across dozens of authentication mechanisms — local username/password, OAuth providers, SAML, and more. You configure a strategy, tell Passport how to serialise a user into the session and deserialise it back out, and Passport handles the rest.

The **Local strategy** (`passport-local`) handles traditional form-based login: it reads a username (or email) and password from the request body, calls a *verify callback* you supply, and passes the result to Passport.

4.2 Configuring the Local Strategy

Stage 1 Configure the strategy and the serialise/deserialise hooks.

`src/auth/passport.ts`

```
import passport from 'passport';
import { Strategy as LocalStrategy } from 'passport-local';
import { getUserRepo } from '../data-source';
import { verifyPassword } from './password';
import { User } from '../entities/User';

/**
 * Configure the Local strategy.
 * passport-local reads `username` and `password` fields from req.body by default.
 * We override `usernameField` to use `email` instead.
 */
passport.use(
    new LocalStrategy(
        { usernameField: 'email' },
        async (email, password, done) => {
            try {
                const repo = getUserRepo();

                // Explicitly select passwordHash — it is excluded by default
                // due to `select: false` on the column (see User entity).
                const user = await repo
                    .createQueryBuilder('user')
                    .addSelect('user.passwordHash')
                    .where('user.email = :email', { email: email.toLowerCase() })
                    .getOne();

                if (!user) {
                    // Use a generic message — never confirm whether the email exists.
                    return done(null, false, { message: 'Invalid credentials.' });
                }

                const isValid = await verifyPassword(password, user.passwordHash);
                if (!isValid) {
                    return done(null, false, { message: 'Invalid credentials.' });
                }

                return done(null, user);
            } catch (err) {
                return done(err);
            }
        }
    )
);

/**
 * Serialise: called after successful login.
```

```
 * Determines what to store in the session — just the user ID.
 */
passport.serializeUser((user, done) => {
    done(null, (user as User).id);
});

/**
 * Deserialise: called on every subsequent authenticated request.
 * Receives the stored ID, fetches the fresh user record from the database.
 */
passport.deserializeUser(async (id: number, done) => {
    try {
        const user = await getUserRepo().findOneBy({ id });
        done(null, user ?? false);
    } catch (err) {
        done(err);
    }
});

export default passport;
```

What is happening? The *verify callback* receives the submitted email and password. It fetches the user from the database (using `addSelect` to force-include `passwordHash`), checks the password, and calls `done` with either the user object on success, or `false` on failure. It never calls `done` with both a user and an error simultaneously — the three-argument signature maps to (error, user, info).

Serialisation and deserialisation separate authentication (happening once at login) from identification (happening on every request). Only the user's ID is stored in the session — a tiny integer rather than the entire user record. On each request, Passport calls `deserializeUser` to exchange that ID for a fresh database record, which it attaches to `req.user`.

4.3 Extending `req.user` in TypeScript

By default, Passport types `req.user` as `Express.User`, which is an empty interface. Augment it so TypeScript knows `req.user` is a `User` entity.

`src/auth/passport.d.ts`

```
import { User } from '../entities/User';

declare global {
    namespace Express {
```

```
        interface User extends import('../entities/User').User {}
    }
}
```

4.4 Wiring Passport into the Express App

Stage 2 Register the session and Passport middleware in the correct order.

src/app.ts

```
import 'reflect-metadata';
import express from 'express';
import session from 'express-session';
import passport from './auth/passport';
import { AppDataSource } from './data-source';
import authRoutes from './routes/auth.routes';
import protectedRoutes from './routes/protected.routes';
import './session.d';

const app = express();

app.use(express.json());

// Session middleware MUST come before passport.session()
app.use(session({
    secret: process.env.SESSION_SECRET ?? 'dev-secret-change-me',
    resave: false,
    saveUninitialized: false,
    name: 'sid',
    cookie: {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'lax',
        maxAge: 30 * 60 * 1000,
    },
}));

// passport.initialize() sets up the req.user property.
// passport.session() reads the session and calls deserializeUser on each request.
app.use(passport.initialize());
app.use(passport.session());

app.use('/auth', authRoutes);
app.use('/api', protectedRoutes);
```

```
AppDataSource.initialize()
    .then(() => {
        app.listen(3000, () => console.log('Server running on http://localhost:3000'));
    })
    .catch((err) => {
        console.error('Database connection failed:', err);
        process.exit(1);
    });
```

Important — Middleware Order: `session()` must be registered before `passport.session()`. Passport's session middleware reads from `req.session`, which must already exist by the time Passport runs. Reversing the order silently breaks session persistence and is a common source of confusing bugs.

4.5 Registration and Login Routes

Stage 3 Build the registration endpoint and the login endpoint. The login route introduces `req.session.regenerate()` to prevent session fixation.

`src/routes/auth.routes.ts`

```
import { Router, Request, Response, NextFunction } from 'express';
import passport from '../auth/passport';
import { getUserRepo } from '../data-source';
import { hashPassword } from '../auth/password';

const router = Router();

// ── POST /auth/register ──────────────────────────────────────────
router.post('/register', async (req: Request, res: Response) => {
    const { email, password } = req.body as { email?: string; password?: string };

    if (!email || !password) {
        res.status(400).json({ error: 'Email and password are required.' });
        return;
    }

    if (password.length < 8) {
        res.status(400).json({ error: 'Password must be at least 8 characters.' });
        return;
    }

    const repo = getUserRepo();
```

```
    const existing = await repo.findOneBy({ email: email.toLowerCase() });
    if (existing) {
        res.status(409).json({ error: 'An account with that email already exists.' });
        return;
    }

    const user = repo.create({
        email: email.toLowerCase(),
        passwordHash: await hashPassword(password),
        isActive: true,
    });

    await repo.save(user);

    // Never return passwordHash in the response.
    res.status(201).json({ id: user.id, email: user.email });
});

// ── POST /auth/login ──────────────────────────────────────────────
router.post('/login', (req: Request, res: Response, next: NextFunction) => {
    passport.authenticate(
        'local',
        (err: Error | null, user: Express.User | false, info: { message: string }) => {
            if (err) { return next(err); }

            if (!user) {
                return res.status(401).json({ error: info?.message ?? 'Authentication
failed.' });
            }

            // Prevent session fixation: regenerate the session ID before
            // writing the user identity into the new session.
            // An attacker who obtained the pre-login session ID cannot
            // use it after this point.
            req.session.regenerate((regenErr) => {
                if (regenErr) { return next(regenErr); }

                // req.login() calls serializeUser and stores the user ID in the session.
                req.login(user, (loginErr) => {
                    if (loginErr) { return next(loginErr); }
                    res.json({ id: user.id, email: user.email });
                });
            });
        }
    )(req, res, next);
});
```

```
// ── POST /auth/logout ──────────────────────────────────────────────
router.post('/logout', (req: Request, res: Response, next: NextFunction) => {
    req.logout((err) => {
        if (err) { return next(err); }
        req.session.destroy(() => {
            res.clearCookie('sid');
            res.json({ message: 'Logged out successfully.' });
        });
    });
});

// ── GET /auth/me ───────────────────────────────────────────────────
router.get('/me', (req: Request, res: Response) => {
    if (!req.isAuthenticated()) {
        res.status(401).json({ error: 'Not authenticated.' });
        return;
    }
    res.json({ id: req.user!.id, email: req.user!.email });
});

export default router;
```

Output:

```
POST /auth/register   body: { "email": "alice@example.com", "password": "securePass1" }
Status: 201
{ "id": 1, "email": "alice@example.com" }

POST /auth/login   body: { "email": "alice@example.com", "password": "securePass1" }
Status: 200
Set-Cookie: sid=s%3AnewSessionId...; Path=/; HttpOnly; SameSite=Lax
{ "id": 1, "email": "alice@example.com" }

POST /auth/login   body: { "email": "alice@example.com", "password": "wrongpassword" }
Status: 401
{ "error": "Invalid credentials." }

GET /auth/me   (with valid session cookie)
{ "id": 1, "email": "alice@example.com" }

GET /auth/me   (without cookie)
Status: 401
{ "error": "Not authenticated." }

POST /auth/logout
{ "message": "Logged out successfully." }
```

Session Fixation — Why `regenerate()` Matters

A session fixation attack works like this: an attacker visits your site and obtains a valid session ID. They then trick a victim into making a request that includes that same session ID — for example, by embedding it in a URL parameter on a phishing page. If your login endpoint writes the user's identity into the *existing* session without changing the ID, the attacker's known session ID now belongs to the authenticated victim. Calling `req.session.regenerate()` immediately before `req.login()` issues a brand-new session ID, so the attacker's old ID becomes useless.

> **Important:** `req.session.regenerate()` copies the session data to a new ID and deletes the old session from the store. After calling it, `req.session` now refers to the *new* session. Call `req.login()` inside the regenerate callback — not before it — or the user identity will be written to the old (now-deleted) session.

---

5. Token-Based Authentication with JWT

5.1 What is a JSON Web Token?

A JSON Web Token (JWT) is a compact, self-contained string composed of three Base64URL-encoded segments separated by dots: `header.payload.signature`.

- **Header** — algorithm and token type: `{ "alg": "HS256", "typ": "JWT" }`
- **Payload** — the claims: any JSON data you choose to include (user ID, email, roles, expiry)
- **Signature** — HMAC of header + payload using a server-side secret, preventing tampering

Because the signature is verifiable without database access, *any* server that knows the secret can validate the token. This is what makes JWT well-suited for horizontally scaled APIs.

Analogy — The Notarised Letter
A JWT is like a letter carrying a notary's stamp. Anyone who knows what the notary's stamp looks like can verify the letter has not been forged or altered since it was issued. You do not need to call the notary on every verification — the stamp itself is proof. If the letter's contents are changed, the stamp no longer matches.

5.2 JWT Helpers

Stage 1 Create typed wrappers around `jsonwebtoken` for signing and verifying tokens.

src/auth/jwt.ts

```
import jwt from 'jsonwebtoken';

const JWT_SECRET = process.env.JWT_SECRET ?? 'dev-jwt-secret-change-me';
const JWT_EXPIRES_IN = '1h';  // Token lifetime

export interface JwtPayload {
    sub: number;    // Subject — the user ID (standard JWT claim name)
    email: string;
    iat?: number;   // Issued At — added automatically by jsonwebtoken
    exp?: number;   // Expiry — added automatically when expiresIn is set
}

/**
 * Signs a JWT containing the user's ID and email.
 * Returns the compact token string.
 */
export function signToken(payload: Omit<JwtPayload, 'iat' | 'exp'>): string {
    return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
}

/**
 * Verifies a JWT string and returns the decoded payload.
 * Throws if the token is invalid, expired, or tampered with.
 */
export function verifyToken(token: string): JwtPayload {
    return jwt.verify(token, JWT_SECRET) as JwtPayload;
}
```

Output:

```
// Quick test in ts-node REPL:
const token = signToken({ sub: 1, email: 'alice@example.com' });
console.log(token);
//
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImVtYWlsIjoiYWxpY2VAZXhhbXBsZS5jb20iLCJpYXQiOjE3MDk0MDAwMDAsImV4cCI6MTcwOTQwMzYwMH0.sOmEsIGnAtUrEhErE

const payload = verifyToken(token);
console.log(payload);
// { sub: 1, email: 'alice@example.com', iat: 1709400000, exp: 1709403600 }
```

What is happening? `jwt.sign()` creates the three-part token. The `sub` claim (subject) is a standard JWT claim name for the entity the token represents — conventionally the user's ID. `jwt.verify()` checks the signature and the `exp` claim simultaneously; if the token has expired or the signature does not match, it throws a `JsonWebTokenError` or `TokenExpiredError` respectively.

5.3 JWT Login Route

Stage 2 Add a token-based login endpoint alongside the Passport session login. Both authenticate against the same User entity and bcrypt logic — only the response format differs.

src/routes/auth.routes.ts (addition)

```
import { signToken } from '../auth/jwt';
import { verifyPassword } from '../auth/password';

// ── POST /auth/token ──────────────────────────────────────────
// Returns a JWT instead of setting a session cookie.
// Designed for API clients (SPAs, mobile apps, third-party consumers).
router.post('/token', async (req: Request, res: Response) => {
    const { email, password } = req.body as { email?: string; password?: string };

    if (!email || !password) {
        res.status(400).json({ error: 'Email and password are required.' });
        return;
    }

    const repo = getUserRepo();
    const user = await repo
        .createQueryBuilder('user')
        .addSelect('user.passwordHash')
        .where('user.email = :email', { email: email.toLowerCase() })
```

```
        .getOne();

    // Always call verifyPassword even when user is not found,
    // to prevent timing-based user enumeration attacks.
    const dummyHash = '$2b$12$invaliddummyhashfortimingprotection00000000000000000000';
    const isValid = user
        ? await verifyPassword(password, user.passwordHash)
        : await verifyPassword(password, dummyHash);  // always runs, always fails

    if (!user || !isValid) {
        res.status(401).json({ error: 'Invalid credentials.' });
        return;
    }

    const token = signToken({ sub: user.id, email: user.email });

    res.json({
        accessToken: token,
        tokenType: 'Bearer',
        expiresIn: 3600,
    });
});
```

Output:

```
POST /auth/token   body: { "email": "alice@example.com", "password": "securePass1" }
{
  "accessToken": "eyJhbGci....",
  "tokenType": "Bearer",
  "expiresIn": 3600
}

POST /auth/token   body: { "email": "nobody@example.com", "password": "anything" }
Status: 401
{ "error": "Invalid credentials." }
```

Key Insight — Timing Attack Mitigation: When a user is not found, naively skipping `verifyPassword` makes the "user not found" path significantly faster than the "wrong password" path. An attacker timing thousands of requests can detect which emails are registered versus which are not. Running `verifyPassword` against a dummy hash on the "not found" branch equalises the response time, preventing this enumeration technique.

6. Auth Guard Middleware

An auth guard is Express middleware that sits in front of protected routes. It checks whether the current request is authenticated and either calls `next()` to allow it through, or terminates the request with a 401 response.

6.1 Session Auth Guard

Passport exposes `req.isAuthenticated()` — a boolean method that returns `true` if `deserializeUser` succeeded and `req.user` is populated.

src/auth/guards.ts

```
import { Request, Response, NextFunction } from 'express';
import { verifyToken, JwtPayload } from './jwt';
import { getUserRepo } from '../data-source';

/**
 * Session guard — protects routes that rely on Passport session authentication.
 * Use this for traditional web application routes.
 */
export function requireSession(req: Request, res: Response, next: NextFunction): void {
    if (req.isAuthenticated()) {
        next();
        return;
    }
    res.status(401).json({ error: 'Authentication required.' });
}

/**
 * JWT guard — protects routes that accept Bearer token authentication.
 * Reads the token from the Authorization header, verifies it,
 * fetches the user, and attaches them to req.user.
 *
 * Clients must send: Authorization: Bearer <token>
 */
export async function requireJwt(
    req: Request,
    res: Response,
    next: NextFunction
): Promise<void> {
    const authHeader = req.headers['authorization'];
```

```
    if (!authHeader?.startsWith('Bearer ')) {
        res.status(401).json({ error: 'Missing or malformed Authorization header.' });
        return;
    }

    const token = authHeader.slice(7);  // Remove 'Bearer ' prefix

    try {
        const payload: JwtPayload = verifyToken(token);

        const user = await getUserRepo().findOneBy({ id: payload.sub });
        if (!user) {
            res.status(401).json({ error: 'User not found.' });
            return;
        }

        // Attach the user to req so downstream route handlers can access it.
        req.user = user;
        next();
    } catch (err: unknown) {
        // Distinguish expired tokens from invalid ones for better client UX.
        if (err instanceof Error && err.name === 'TokenExpiredError') {
            res.status(401).json({ error: 'Token has expired. Please log in again.' });
        } else {
            res.status(401).json({ error: 'Invalid token.' });
        }
    }
}
```

6.2 Using Guards on Protected Routes

src/routes/protected.routes.ts

```
import { Router, Request, Response } from 'express';
import { requireSession, requireJwt } from '../auth/guards';

const router = Router();

// Protected by session cookie — suitable for server-rendered pages or
// same-origin API calls from a browser.
router.get('/profile', requireSession, (req: Request, res: Response) => {
    res.json({
        message: 'Session-authenticated profile data.',
        user: { id: req.user!.id, email: req.user!.email },
    });
```

```
});

// Protected by JWT Bearer token — suitable for API clients, mobile apps.
router.get('/dashboard', requireJwt, (req: Request, res: Response) => {
    res.json({
        message: 'JWT-authenticated dashboard data.',
        user: { id: req.user!.id, email: req.user!.email },
    });
});

// A route can accept EITHER authentication method using a helper that
// tries both guards in sequence.
router.get('/data', tryAnyAuth, (req: Request, res: Response) => {
    res.json({
        message: 'Accessible via session or JWT.',
        user: { id: req.user!.id, email: req.user!.email },
    });
});

/**
 * Tries session auth first; if unauthenticated, tries JWT.
 * Only rejects the request if both fail.
 */
async function tryAnyAuth(req: Request, res: Response, next: (err?: unknown) => void) {
    if (req.isAuthenticated()) {
        return next();
    }
    await requireJwt(req, res, next);
}

export default router;
```

Output:

```
GET /api/profile   (with valid session cookie)
{
  "message": "Session-authenticated profile data.",
  "user": { "id": 1, "email": "alice@example.com" }
}

GET /api/profile   (no cookie)
Status: 401
{ "error": "Authentication required." }

GET /api/dashboard   (Authorization: Bearer eyJhbGci....)
{
  "message": "JWT-authenticated dashboard data.",
```

```
  "user": { "id": 1, "email": "alice@example.com" }
}

GET /api/dashboard   (expired token)
Status: 401
{ "error": "Token has expired. Please log in again." }

GET /api/dashboard   (no header)
Status: 401
{ "error": "Missing or malformed Authorization header." }
```

What is happening? Each guard is a standard Express middleware function — a function that accepts `(req, res, next)`. Placing the guard as the second argument in a route definition (`router.get('/profile', requireSession, handler)`) makes Express call it before the handler. If the guard calls `next()`, Express proceeds to the handler. If the guard calls `res.json()` directly, the request is terminated at the guard and the handler never runs.

> **Key Insight — Fetch vs Lookup on JWT Verification:** The JWT guard fetches the user from the database on every request rather than trusting the payload directly. This has a cost — one database query per authenticated request — but it ensures that deactivated or deleted users cannot continue to use tokens issued before their removal. Whether to pay this cost or trust the payload entirely is an architectural trade-off: if instant revocation is not required and performance is critical, skipping the fetch and using `payload.sub` directly is a valid choice. If you use this approach, lower the token lifetime (e.g., 15 minutes) and implement a refresh token flow.

7. Centralised Error Handling for Auth

Authentication errors should be handled by a dedicated Express error middleware so that uncaught errors in async guards do not silently crash routes. This connects naturally to the error-handling patterns that will be covered in detail in Section 14.

`src/app.ts (addition — register last)`

```
import { Request, Response, NextFunction } from 'express';

// Must be registered after all routes.
// Four-parameter signature tells Express this is an error handler.
```

```
app.use((err: Error, req: Request, res: Response, _next: NextFunction) => {
    console.error(`[${new Date().toISOString()}] Unhandled error:`, err.message);

    // Avoid leaking stack traces in production.
    const message = process.env.NODE_ENV === 'production'
        ? 'An internal server error occurred.'
        : err.message;

    res.status(500).json({ error: message });
});
```

Output:

```
// When a database connection drops mid-request:
[2026-03-01T10:00:00.000Z] Unhandled error: Connection terminated unexpectedly

// Response to client (production):
Status: 500
{ "error": "An internal server error occurred." }
```

8. Authentication Strategy Summary

Characteristic	Session + Passport	JWT
State location	Server (session store)	Client (token string)
Instant revocation	Yes — delete session from store	No — token valid until expiry (unless blocklist added)
Horizontal scaling	Requires shared store (Redis)	Stateless — any instance verifies
Client type	Browsers with cookie support	Any client; common for APIs and mobile
CSRF risk	Yes — cookies sent automatically; mitigate with `SameSite` and CSRF tokens	No — tokens are not sent automatically by browsers

XSS risk	Lower if `HttpOnly` cookie	Higher if token stored in `localStorage`
Token lifetime	Controlled server-side via session expiry	Embedded in token — client cannot extend it
Database hit per request	Yes (deserializeUser)	Optional (see Section 6 discussion)

Summary

Concept	Key Point
bcrypt	Use for password hashing — built-in salt and configurable cost factor make brute-force and rainbow table attacks impractical. Never use fast hashes (MD5, SHA-256) for passwords.
`@Column({ select: false })`	Excludes sensitive columns from default query results. Use `addSelect()` in QueryBuilder to explicitly include them when needed.
Passport Local strategy	Handles username/password authentication via a verify callback. Configure `usernameField` to use email instead of the default `username`.
Serialise / Deserialise	Serialise stores just the user ID in the session; deserialise exchanges that ID for a fresh database record on each request.
`req.session.regenerate()`	Issues a new session ID immediately before writing user identity into the session. Prevents session fixation attacks. Must be called before `req.login()`.

JWT structure	Three Base64URL segments: header, payload, signature. Any party with the secret can verify the signature without database access.
`jwt.sign()`	Creates a signed token. Use `sub` for user ID (standard claim). Set a short `expiresIn`.
`jwt.verify()`	Throws `TokenExpiredError` or `JsonWebTokenError` on failure — catch and handle both explicitly.
Auth guard middleware	A standard Express middleware function that calls `next()` on success and sends a 401 on failure. Applied as a route argument before the handler.
Timing attack prevention	Always run the full password verification path, even when the user is not found. Skipping it leaks a timing signal that reveals which emails are registered.
Generic error messages	Return "Invalid credentials" for both wrong email and wrong password. Never confirm whether an email address is registered.
Middleware order	`session()` → `passport.initialize()` → `passport.session()`. Deviating from this order silently breaks session persistence.

Interview Questions

1. **Why is bcrypt preferred over SHA-256 for password storage?**
   Expected answer: SHA-256 is a fast general-purpose hash designed for throughput. Fast hashes allow an attacker to attempt billions of candidates per second on modern GPUs. bcrypt is intentionally slow — its work factor controls the number of iterations, making each

attempt take 100+ milliseconds. It also embeds a random salt in the output automatically, making pre-computed rainbow table attacks useless. The cost of bcrypt for legitimate logins (one hash per login) is negligible, while the cost for brute-force attacks (billions of hashes) is prohibitive.

2. **What is session fixation, and how does `req.session.regenerate()` prevent it?**
   Expected answer: Session fixation is an attack where an adversary plants a known session ID on a victim's browser before login. If the server then associates user identity with that existing (known) session ID, the attacker can use their known ID to access the victim's authenticated session. Calling `req.session.regenerate()` after the user is verified but before their identity is stored issues a brand-new, cryptographically random session ID. The attacker's planted ID is now invalid, so they gain nothing.

3. **What is the difference between `passport.serializeUser` and `passport.deserializeUser`, and why does `serializeUser` typically only store the user's ID?**
   Expected answer: `serializeUser` runs once, at login, and determines what is saved into the session. `deserializeUser` runs on every subsequent request and retrieves the full user object using what was saved. Storing only the ID keeps the session small and avoids stale data — if a user's email or role changes between requests, the `deserializeUser` fetch from the database picks up the latest record automatically. Storing the whole user object in the session would risk serving outdated profile information.

4. **Describe the three parts of a JWT and explain what the signature section prevents.**
   Expected answer: A JWT has a header (algorithm and type), a payload (claims — arbitrary JSON data), and a signature (HMAC of header + payload using a server secret). The signature prevents tampering: if any byte in the header or payload changes, the signature no longer verifies against the secret. An attacker cannot modify the payload (e.g., changing their user ID or role) without knowing the secret, and they cannot forge a valid signature without it.

5. **What is the main trade-off between session-based and JWT-based authentication when it comes to revocation?**
   Expected answer: Session-based authentication allows instant revocation — deleting a session record from the store immediately invalidates it. JWT is stateless; once issued, a token remains valid until its `exp` claim passes, regardless of whether the user's account has been deactivated. To achieve revocation with JWT, you must implement a token blocklist (a store of

invalidated token IDs), which reintroduces server-side state and partially negates the stateless benefit. The practical mitigation is to issue short-lived access tokens (15 minutes) paired with a refresh token flow.

6. **Why should you run `bcrypt.compare()` even when you know the user does not exist in the database?**
   Expected answer: To prevent timing-based user enumeration. If the "user not found" path skips the hash comparison and returns immediately, it is measurably faster than the "wrong password" path, which runs bcrypt. An attacker can detect this difference across many requests and determine which email addresses are registered, without ever seeing a login error message. Running a dummy `bcrypt.compare()` on the "not found" branch equalises the response time, removing this information channel.

7. **What does `@Column({ select: false })` do in TypeORM, and when would you need to override it?**
   Expected answer: It excludes the column from the results of standard repository `find*` methods, preventing sensitive data from accidentally appearing in query results. You override it by using QueryBuilder's `addSelect('entity.column')` method — which explicitly names the column in the SQL `SELECT`. The typical use case is the password hash column: excluded from all general queries that return user data to clients, but explicitly selected during login verification.

8. **What is the correct Express middleware registration order for Passport with sessions, and what goes wrong if the order is incorrect?**
   Expected answer: The correct order is: `express-session` middleware first, then `passport.initialize()`, then `passport.session()`. `passport.session()` reads from `req.session`, which is populated by the session middleware. If Passport's session middleware runs before the session middleware, `req.session` is undefined and Passport silently fails to restore session state. The result is that every request appears unauthenticated even after a successful login, which is a non-obvious bug to diagnose.

Practical Assignment — Full Auth Flow with Dual Strategy

Objective: Build a complete authentication system in TypeScript that supports both session-based and JWT-based auth against a PostgreSQL database via TypeORM, with protected routes accessible under each strategy.

1. Initialise a new TypeScript project and install all required packages:
```
npm install express express-session passport passport-local bcrypt jsonwebtoken typeorm reflect-metadata pg
```
```
npm install --save-dev typescript ts-node-dev @types/express @types/express-session @types/passport @types/passport-local @types/bcrypt @types/jsonwebtoken @types/pg
```
2. Create the `User` entity as shown in Section 3.1. Add one extra column: `role` typed as a TypeScript union `'user' | 'admin'`, using a TypeORM enum column with a default value of `'user'`.
3. Implement `hashPassword` and `verifyPassword` helpers as shown in Section 3.2. Use a salt round of `12`.
4. Configure the Passport Local strategy and serialise/deserialise hooks as in Section 4.2. Remember to explicitly select `passwordHash` using `addSelect`.
5. Implement `POST /auth/register` (validates input, hashes password, saves user, returns `{ id, email }`) and `POST /auth/login` (Passport session auth with `req.session.regenerate()` before `req.login()`).
6. Implement `POST /auth/token` returning a JWT. Include the user's `role` in the JWT payload.
7. Write two guard middleware functions: `requireSession` and `requireJwt` as in Section 6.1.
8. Add three protected routes:
   - `GET /api/me/session` — session guard only, returns user profile.
   - `GET /api/me/jwt` — JWT guard only, returns user profile and role from the token payload.
   - `GET /api/admin` — JWT guard, additionally checks that `req.user.role === 'admin'` and returns 403 if not.
9. Test the full flow using curl or a REST client: register, session login, access `/api/me/session`, token login, access `/api/me/jwt`, attempt `/api/admin` as a non-admin user.

Expected Output Sequence:

Output:

```
POST /auth/register   { "email": "bob@example.com", "password": "password123" }
Status: 201
{ "id": 1, "email": "bob@example.com" }

POST /auth/login   { "email": "bob@example.com", "password": "password123" }
Status: 200   Set-Cookie: sid=...
{ "id": 1, "email": "bob@example.com" }

GET /api/me/session   (with session cookie)
{ "id": 1, "email": "bob@example.com", "role": "user" }

POST /auth/token   { "email": "bob@example.com", "password": "password123" }
{ "accessToken": "eyJ...", "tokenType": "Bearer", "expiresIn": 3600 }

GET /api/me/jwt   Authorization: Bearer eyJ...
{ "id": 1, "email": "bob@example.com", "role": "user" }

GET /api/admin   Authorization: Bearer eyJ...  (bob is role: 'user')
Status: 403
{ "error": "Insufficient permissions." }

GET /api/me/session   (no cookie)
Status: 401
{ "error": "Authentication required." }
```

Bonus Challenge: Implement a token refresh flow. Add a `POST /auth/refresh` endpoint that accepts a long-lived *refresh token* (valid for 7 days, signed with a separate `REFRESH_TOKEN_SECRET`) issued alongside the access token. The endpoint verifies the refresh token, fetches the user, and issues a new short-lived access token (15 minutes). Store issued refresh token IDs in a `refresh_tokens` table (a new TypeORM entity with `tokenId`, `userId`, and `expiresAt` columns) and delete the record on use, so each refresh token is single-use.